

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a6e5853c50ee4dfd3.jsonl`  
- SHA-256: `3b05f1582b1a9f470bb4b1d468f23ce2fd3d3e9841fc317a528338444166b0cd`  
- Source modified: `2026-05-31T04:19:42+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:18:43.440Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:
URL: https://www.businesswaretech.com/blog/research-best-ai-services-for-automatic-invoice-processing
Title: AWS Textract vs Google, Azure & GPT-4o: Invoice Extraction Benchmark
Found via: Approval-gated paid/cloud fallbacks & data terms (cost/tradeoffs) search

Task

- 1. Use WebFetch to retrieve the page content.
- 2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
- 3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
- 4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:18:45.881Z)

I'll fetch the source and extract key claims. Let me first load the WebFetch tool schema.

3. assistant (2026-05-31T04:18:48.617Z)

I'll fetch the page content now.

4. user (2026-05-31T04:19:03.018Z)

Invoice Extraction Benchmark: Key Factual Claims

Accuracy Results (Field-Level)

Service	Field Accuracy	Source Quote
----- --- ---		
GPT-4o + OCR	98.0%	"GPT-4o + OCR (GPTt), with the highest accuracy across all structured fields"
Azure Document Intelligence	93.0%	"Azure: **93%** , AWS: **78%**"
GPT-4o (Image Input)	90.5%	"strong field accuracy (90.5%)"
Google Document AI	82.0%	"Google Document AI field accuracy: **~82%**"
AWS Textract	78.0%	"Textract field accuracy: **~78%**"

Line-Item Extraction Performance

Service	Score	Quote
Azure	87.0%	"Azure was the top performer, consistently extracting tables"
AWS Textract	82.0%	"Textract: **82%**"
GPT-4o (Image)	63.0%	"weak on table/line-item extraction"
GPT-4o + OCR	57.0%	Results table shows 57%
Google Document AI	40.0%	"Google Document AI underperformed...40%"

Speed & Cost

Processing Time per Page:

- AWS Textract: 2.9 ± 0.2 seconds
- Google Document AI: 3.8 ± 0.2 seconds
- Azure Document Intelligence: 4.3 ± 0.2 seconds
- GPT-4o (Image Input): 16.9 ± 1.9 seconds
- GPT-4o + OCR: 33.0 ± 2.3 seconds

Pricing: ~\$10 per 1,000 pages for both (AWS and Google); identical for Azure

Test Methodology

Dataset: "scanned digital invoices...JPG, PNG, PDF (without a text layer)...at least 3 different types of layouts...invoices issued from 1971 to 2020"

Services Tested:

1. Amazon Analyze Expense API
2. Azure AI Document Intelligence - Invoice Prebuilt Model
3. Google Document AI - Invoice Parser
4. GPT-4o API - text input with 3rd party OCR
5. GPT-4o API - image input

Specific Fields Evaluated: "vendor names, invoice totals, and itemized line entries"

Data Privacy/Retention Terms

Not addressed in the provided content.

Self-Hosting Options

Not discussed in the article.

Publication Details

- **Date:** January 2025
- **Author:** Businessware Technologies (implied through domain/footer)
- **Original Research:** Yes?indicated as proprietary benchmark testing ("we conducted a detailed benchmark")

Key Winner Claims

Overall: "The most effective services...are Azure AI Document Intelligence and GPT-4o API with 3rd party OCR"

Maximum accuracy: GPT-4o + OCR at 98% (newer invoices only)

Best cloud-native: Azure (93% field accuracy vs. Textract's 78%)

5. assistant (2026-05-31T04:19:05.817Z)

I have the content. Let me do a follow-up fetch to verify a couple of specific numbers and the

methodology detail (sample size, and the GPT-4o + OCR table-extraction figure, which appeared inconsistent across the two summaries) to ensure my quotes are accurate.

6. user (2026-05-31T04:19:13.645Z)

Benchmark Article Analysis

(1) Dataset Size:

The article does not specify an exact number of invoices in the test dataset. It states: "We have put together a dataset containing scanned digital invoices in the following formats: JPG, PNG, PDF" and mentions "at least 3 different types of layouts," but no total count is provided.

(2) Best Service Conclusion:

"The most effective services for detecting invoice fields are Azure AI Document Intelligence and GPT-4o API with 3rd party OCR"

(3) GPT-4o + OCR Field Accuracy:

"GPT-4o + OCR (GPTt), with the highest accuracy across all structured fields"
(The 98% figure appears in a table but is not quoted in a full sentence in the text.)

(4) Azure vs AWS Field Accuracy Comparison:

"Field accuracy: 93% vs AWS's 78%"

(5) GPT-4o Weakness on Tables:

"GPT-4o-based models struggled with structured tables despite excellent field accuracy" and
"Line-Item Score...GPT-4o (Image Input) 63.0%"

(6) Privacy/Data Retention:

Not mentioned in the article.

7. assistant (2026-05-31T04:19:22.044Z)

I now have a clear, verified picture of the source. Let me synthesize the findings.

This is an original benchmark study by Businessware Technologies (a tech consultancy/blog) comparing four cloud document-AI services plus GPT-4o on invoice extraction. It is directly relevant to the research question's paid/cloud fallback section, though it covers invoices (not bank statements) and does not address the data-handling/privacy terms that muneem's sensitivity constraint cares most about.

Key relevance and caveats:

- The benchmark covers invoices, not bank-statement transaction tables ? adjacent but not identical to muneem's target documents.
- It is a vendor/consultancy blog presenting its own (proprietary, non-reproducible) benchmark; sample size and exact methodology are underspecified, which limits rigor.
- It crucially does NOT address data privacy, retention, or training-use terms ? the very thing that matters for muneem's "approval-gated, sensitive" framing. Worth flagging as a gap.
- The most decision-relevant finding for muneem: cloud document-AI services (Azure) substantially outperform general-purpose VLMs (GPT-4o image input) on table/line-item structure extraction (87% vs 63%), even though the VLM matches or leads on individual field accuracy. This bears directly on the research question's core tension (prompt-driven VLM vs OCR+table-structure pipeline for dense tables).

8. user (2026-05-31T04:19:32.243Z)

Structured output provided successfully

9. assistant (2026-05-31T04:19:42.882Z)

Source processed. Summary of what I found and how it bears on the research question:

Source: Businessware Technologies blog ? an original (self-described "we conducted a detailed benchmark") invoice-extraction benchmark comparing AWS Textract (Analyze Expense),

Azure AI Document Intelligence (Invoice prebuilt), Google Document AI (Invoice Parser), and GPT-4o (text+OCR and image-input). Published ~January 2025. Rated **blog** quality: it presents proprietary, non-reproducible numbers (no stated sample size, vague methodology), so figures should be treated as indicative, not authoritative.

Most decision-relevant takeaway for muneem: This directly informs the research question's central tension ? prompt-driven VLM vs. classic OCR+table-structure pipeline for dense tables. The benchmark shows dedicated document-AI services (Azure ~87%, Textract ~82%) substantially **out-extract** the general-purpose VLM on table/line-item structure (GPT-4o image 63%), even though the VLM matches or leads on isolated scalar fields (GPT-4o+OCR 98%, Azure 93% field accuracy). This is evidence that VLMs are strong at reading individual values but weaker at reconstructing multi-row table structure ? exactly the failure mode (dropped/duplicated rows) the research question flags, and an argument for keeping a structure-aware approach (or balance-reconciliation verification) rather than trusting a VLM's whole-table JSON.

Caveats I flagged in the claims:

- Domain mismatch: this is **invoices**, not bank-statement transaction tables ? adjacent but not identical to muneem's targets.
- It says **nothing** about data privacy, retention, or training-use terms ? the single most important dimension for muneem's "sensitive / approval-gated" cloud framing. This source does not help close that gap.
- Cost anchor extracted: ~\$10 / 1,000 pages (~\$0.01/page) for AWS/Google/Azure.

All findings returned via the StructuredOutput tool (5 claims, 2 central / 2 supporting / 1 tangential).